LiDAR Obstacle Segmentation Pipelines

`coda_pipeline.py` (offline, CODa) and `live_pipeline.py` (live, ROS 2)

October 21, 2026

Contents

1 Overview **2**

2 The shared algorithm **2**

2.1 Preprocessing . . . 2

2.2 Ray ground filter . . . 3

2.3 Euclidean clustering . . . 3

2.4 Box fitting . . . 3

3 Pipeline 1: `coda_pipeline.py` (offline, with scoring) **4**

3.1 What it does . . . 4

3.2 Expected data layout . . . 4

3.3 How to run . . . 4

3.4 Reading the output . . . 5

4 Pipeline 2: `live_pipeline.py` (ROS 2, live LiDAR) **5**

4.1 What it does . . . 5

4.2 How to run . . . 6

4.3 Tuning without driving: replay a bag . . . 6

4.4 Running on a Jetson . . . 6

4.5 Status . . . 6

5 Tuning parameters **7**

5.1 Symptom-to-parameter guide . . . 8

5.2 Suggested tuning order . . . 8

6 Limitations **9**

1 Overview

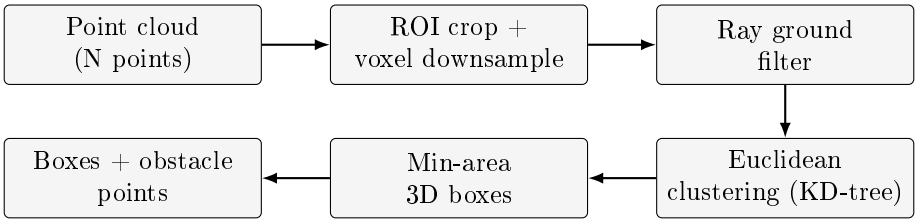
There are two single-file scripts. They share the same perception core and differ only in where point clouds come from and what happens to the result.

- `coda_pipeline.py` reads frames of the CODa dataset from disk, runs the pipeline, *scores* the output against CODa's own labels (ground segmentation and 3D boxes), and writes top-down images and optional JSON. It is the tool for developing and tuning the algorithm.
- `live_pipeline.py` is a ROS 2 node. It subscribes to a `sensor_msgs/PointCloud2` topic published by a real LiDAR driver, runs the same core on every incoming frame, and publishes boxes and obstacle points for RViz or downstream code. There is no ground truth, so it does no scoring.

The core algorithm, in order, is:

1. Preprocessing: remove invalid points, crop to a region of interest, voxel-downsample.
2. Ray-based ground filtering.
3. Euclidean clustering of the non-ground points with a KD-tree.
4. Minimum-area oriented 3D bounding box for each cluster.

The conventions are: x forward, y left, z up (ROS REP-103, identical to CODa), all in the sensor frame, in metres.



2 The shared algorithm

All of this lives in `segment(pts, a)`, which takes an array of points (only the first three columns, x, y, z, are used) and an options object `a` holding the parameters of Section 5. It returns the kept points, a per-point ground mask, a per-point cluster label, the list of boxes, and timings.

2.1 Preprocessing

1. Points that are non-finite or exactly $(0, 0, 0)$ (no-return placeholders) are dropped.
2. **Region of interest.** A point is kept if its horizontal range $r = \sqrt{x^2 + y^2}$ lies in (`min_range`,`max_range`) *and* it lies inside the box given by `roi_x`, `roi_y`, `roi_z` (all in the sensor frame; z is relative to the sensor, not the ground).
3. **Voxel downsampling.** Each point is hashed to an integer voxel of side `voxel`; one point per occupied voxel is kept. This is done *before* the ground filter because it is the cheapest way to cut the point count. A side effect is that all later point-count thresholds (`min_size`) refer to downsampled points, not raw returns.

2.2 Ray ground filter

The points are binned by azimuth into 360/`ray_div` rays (by default 360 rays of 1°). Within each ray, points are sorted from near to far by horizontal range r and scanned in a single pass. For point i with range r_i and height z_i:

- The reference ground level is $z_{\text{ref}} = -$`sensor_height` if the sensor height is given. Otherwise it is the 5th percentile of z among points within 10 m (or of all points if fewer than 100 are that close).
- **Global cone:** the point is compatible with the ground if $|z_i - z_{\text{ref}}| \leq \max(\tan\theta_g\, r_i,\ h_{\min})$.
- **Local cone:** relative to the previous point in the ray, $|z_i - z_{i-1}| \leq \max(\tan\theta\, \Delta r,\ h_{\min})$, where θ is the *initial* slope `init_slope_deg` for the first point of a ray and the local slope otherwise.
- **Anti-creep anchor:** the same slope test is also applied against the last accepted ground "anchor" point, so a long gentle ramp of obstacle points cannot be walked up one small step at a time. The anchor only advances along a real slope.
- **Recovery:** if the previous point was non-ground, a point is re-admitted as ground only if it is inside the global cone and consistent with the anchor. A point that fails the local test can still be accepted if it passes the global cone and the range gap Δr is at least `reclass_dist` (0.2 m; this one is set in the code, not exposed as a flag).

In `segment()`, `slope_deg` is used for both θ_g and the local slope.

After filtering, the ground height z_{ground} is estimated as the median z of ground points within 10 m (falling back to all ground points, then to the 5th percentile of all points, then to 0). Obstacle candidates are the non-ground points with $z - z_{\text{ground}} <$ `max_height`.

2.3 Euclidean clustering

Obstacle candidates are clustered by connectivity: two points are linked if they are closer than `eps`, and clusters are the connected components. The default `"fast"` method builds a `cKDTree`, gets all pairs within `eps` in one call, and runs `connected_components` on the resulting sparse graph. A literal region-growing version (processed flags and a stack) is also in the file for reference but is slower. Clusters with fewer than `min_size` points are discarded.

2.4 Box fitting

For each cluster:

- The x, y coordinates give a convex hull. A minimum-area rectangle is found by testing each hull-edge direction and keeping the orientation with the smallest enclosing area.
- The longer side is the length l, the shorter is the width w, and the yaw is in $[-\pi/2, \pi/2)$.
- Height is $z_{\max} - z_{\min}$ of the cluster and the centre height is the midpoint.
- A cluster whose longer side exceeds `max_box` is rejected (walls, hedges, building facades).

Each box is a dictionary with `cx, cy, cz, l, w, h, yaw`, a running `id`, and `n`, the number of points in the cluster. Boxes have yaw only (no roll or pitch).

3 Pipeline 1: `coda_pipeline.py` (offline, with scoring)

3.1 What it does

1. **Select frames.** If CODa 3D box annotations exist for the sequence, only annotated frames are used (override with `-all-frames`). The list is then sliced as `frames[start::step][:count]`.
2. **Run the core** (Section 2) on each frame and record timings.
3. **Score the ground filter** against `3d_semantic` where available. Semantic id 0 is non-terrain (objects, walls); ids 1–23 except 20 are terrain types and count as ground; ids 20 (stairs) and 24 (unknown) are ignored. With TP = predicted ground that is ground, FP = predicted ground that is object, FN = ground predicted as non-ground, it reports precision, recall and F1.
4. **Score the boxes** against CODa 3D boxes. A CODa box counts only if it is inside the ROI and contains at least `min_gt_pts` downsampled points. It is "found" if at least 50% of those points fall in some cluster. For found objects the centre error between our box (the cluster that owns most of the points) and the CODa box is recorded. Results are broken down by class.
5. **Write outputs:** top-down PNGs for the first `-png` frames and, with `-dump`, CODa-style JSON detections.

3.2 Expected data layout

```
<root>/
  3d_raw/os1/<seq>/3d_raw_os1_<seq>_<frame>.bin      float32 (N,4): x y z intensity
  3d_comp/os1/<seq>/3d_comp_os1_<seq>_<frame>.bin    (ego-compensated; used first if --
    cloud comp)
  3d_bbox/os1/<seq>/3d_bbox_os1_<seq>_<frame>.json   CODa 3D box labels
  3d_semantic/os1/<seq>/3d_semantic_os1_<seq>_<frame>.bin   uint8 per point
```

Only the point clouds are required. Boxes and semantic labels are optional; the matching scores are skipped if they are missing (only about 5K frames have semantic labels).

3.3 How to run

Requirements: `numpy`, `scipy`, `matplotlib`; `numba` is optional and speeds up the ground filter.

```
python3 coda_pipeline.py --root <folder with 3d_raw, 3d_bbox, ...> \
    --seq 0 --count 20 --step 10 --png 5
```

Useful variations:

```
# use all frames, not just annotated ones
python3 coda_pipeline.py --root <root> --seq 0 --all-frames --count 50 --step 5

# known sensor height, coarser voxels for speed, save detections as JSON
python3 coda_pipeline.py --root <root> --seq 0 --sensor-height 0.9 --voxel 0.15 --dump

# ignore the rectangular ROI, use only the circular range limit
python3 coda_pipeline.py --root <root> --seq 0 --no-roi --max-range 40
```

3.4 Reading the output

The console prints the average point counts through the stages (raw → ROI → voxel), average milliseconds for ROI+voxel, ground filter and clustering+boxes, and boxes per frame. Then, if labels exist:

- **Ground precision/recall/F1.** Low precision means obstacle points are being swallowed as ground. Low recall means floor is leaking into obstacles.
- **Boxes found** as a fraction of CODa boxes, per class (top 12 classes), and the median/mean centre error. Our box only covers the visible side of an object, so some centre error is expected.

In the PNGs: grey = ground, blue = non-ground not used, coloured = clusters, green = our boxes (with a heading tick), red dashed = CODa ground truth, dotted black = ROI.

4 Pipeline 2: `live_pipeline.py` (ROS 2, live LiDAR)

4.1 What it does

A node named `lidar_pipeline_live` subscribes to a PointCloud2 topic. For every message it:

1. Converts the message to an $N \times 3$ float array with `cloud_to_xyz()`. It reads x, y, z by their declared field offsets and handles row padding, so it does not depend on the driver's exact layout.
2. Calls `segment()` (Section 2).
3. Publishes the boxes and the clustered obstacle points, with the *same header* (frame and timestamp) as the input cloud, so they align in RViz.

Published topics.

Topic	Type	Content
`/detections/boxes`	`visualization_msgs/MarkerArray`	One green semi-transparent cube per cluster (size $l \times w \times h$, yaw about z). A `DELETEALL` marker is sent first each frame, and each marker has a 0.2 s lifetime so stale boxes disappear if the node stalls.
`/detections/obstacles`	`sensor_msgs/PointCloud2`	The downsampled points that belong to a kept cluster.

Design choices for real-time use.

- *QoS:* best-effort reliability, keep-last, depth 1, like a sensor stream. If processing is slower than the LiDAR, old frames are dropped and the newest is processed instead of building a backlog.
- *Numba warm-up:* the ground-filter kernel is compiled once at start-up on a dummy cloud, so the first real frame does not stall.

- *Parameters:* every flag of the offline script that matters is a ROS parameter (Section 5), set on the command line.
- The log prints the number of points, boxes and milliseconds per frame (throttled to every 2 s). If the milliseconds exceed the LiDAR period (about 100 ms at 10 Hz), frames are being dropped.

4.2 How to run

Requirements: ROS 2 (tested design for Humble: `rclpy`, `sensor_msgs_py`), plus `numpy`, `scipy`; `numba` optional but recommended.

```
source /opt/ros/humble/setup.bash
# 1. start your LiDAR driver (e.g. the Ouster driver) and check the topic:
ros2 topic list
ros2 topic hz /ouster/points
# 2. run the node
python3 live_pipeline.py --ros-args -p topic:=/ouster/points -p sensor_height:=0.9
# list parameters use ROS array syntax:
python3 live_pipeline.py --ros-args -p topic:=/ouster/points \
    -p roi_x:="[-5.0, 25.0]" -p voxel:=0.15
```

In RViz, set the fixed frame to the cloud's `frame_id` and add `/detections/boxes` (MarkerArray) and `/detections/obstacles` (PointCloud2).

4.3 Tuning without driving: replay a bag

Record a short bag once and loop it while you tune:

```
ros2 bag record /ouster/points
ros2 bag play <bag> --loop
```

4.4 Running on a Jetson

- JetPack 6 (Ubuntu 22.04) supports ROS 2 Humble directly. JetPack 5 (Ubuntu 20.04) is easier with a Docker container.
- Install `numpy scipy numba` with pip.
- The pipeline is CPU-only. Enable full clocks with `sudo nvpmodel -m 0 \&\& sudo jetson_clocks`.
- If it is too slow, first raise `voxel` (for example 0.2) or shrink the ROI. The first start is slower while numba compiles and caches.

4.5 Status

The algorithm is the same code that was validated on CODa. The conversion and pipeline were tested on a synthetic cloud (a car-sized and a pedestrian-sized object on flat ground) and both objects were boxed correctly. The ROS plumbing (subscriber, QoS, markers, frames) has **not** been run against a real LiDAR yet, so expect some plumbing fixes and parameter tuning on first contact.

5 Tuning parameters

Offline flags use hyphens (`-roi-x`); the live node uses the same names with underscores as ROS parameters (`roi_x`). Defaults below are the same in both unless noted. The defaults come from CODa's Ouster setup and are a starting point for other sensors.

Parameter	Default	Stage	Effect and when to change
`min_range`	0.8 m	ROI	Drops points closer than this horizontally. Raise it to remove returns from the robot body or mount.
`max_range`	30 m	ROI	Circular range limit. Lower it for speed; far points are sparse and cluster poorly anyway.
`roi_x`, `roi_y`	$[-10, 30]$, $[-15, 15]$ m	ROI	Rectangle in the sensor frame (x forward, y left). Shape it to the area you care about, such as a corridor ahead. Smaller means faster. `-no-roi` (offline) disables it.
`roi_z`	$[-3, 3]$ m	ROI	Height range *relative to the sensor*. It must include the ground: if the sensor is mounted higher than 3 m, widen the lower bound or the ground is cropped away and the filter fails.
`voxel`	0.1 m	Pre	Downsampling cell size. Larger is faster and coarser. It changes point density, so adjust `min_size` and `eps` with it. 0 turns it off.
`sensor_height`	None (live: 0.0)	Ground	LiDAR height above ground in metres. Set it when you know it. None (live: 0.0) means estimate from the 5th percentile of nearby z, which can be biased on slopes or when little ground is visible.
`ray_div`	1.0°	Ground	Azimuth bin width. Smaller gives more rays with fewer points each (thin rays make the scan noisy); larger gives coarser angular resolution.
`slope_deg`	10°	Ground	Maximum ground slope for both the global and local tests. Raise it for ramps and uneven terrain; too high and low obstacles and curbs are absorbed into the ground. Lower it if floor leaks into obstacles on flat ground.
`init_slope_deg`	3°	Ground	Slope tolerance for the first point of each ray (relative to ground below the sensor). Keep it small so a ray cannot start on an obstacle.
`min_h`	0.15 m	Ground	Height tolerance floor: no threshold goes below this. It absorbs measurement noise but also hides obstacles lower than about this. Raise it on noisy sensors or rough ground; lower it to catch small obstacles.
`reclass_dist`	0.2 m	Ground	Range gap that allows a point to be re-admitted as ground. Set in the code (argument of `ray_ground_mask`), not exposed as a flag.

Parameter	Default	Stage	Effect and when to change
`max_height`	2.5 m	Obstacles	Non-ground points higher than this above the estimated ground are ignored (overhangs, tree canopy, signs).
`eps`	0.5 m	Cluster	Linking distance. Too small splits one object into pieces (especially far, sparse ones); too large merges neighbours (a person next to a wall).
`min_size`	8 pts	Cluster	Minimum cluster size in *downsampled* points. Raise it to suppress noise blobs; lower it to detect small or distant objects. Lower it if you raise `voxel`.
`max_box`	12 m	Boxes	Reject clusters whose longer side exceeds this (walls, facades).
`min_gt_pts`	8	Scoring	Offline only. Ignore CODa boxes with fewer downsampled points inside, so unseen objects do not count as misses.

Offline-only controls: `-root` (required), `-seq`, `-cloud comp|raw`, `-start`, `-count`, `-step`, `-all-frames`, `-png N`, `-out DIR`, `-dump`.
Live-only controls: `topic` (input PointCloud2 topic, default `/ouster/points`).

5.1 Symptom-to-parameter guide

Symptom	Things to try
Floor shows up as obstacles	Set `sensor_height`; raise `min_h` or `slope_deg`; check the sensor is level (a tilted mount breaks the cone tests, so transform into a level frame first).
Obstacles or curbs marked as ground	Lower `slope_deg` and `min_h`; set `sensor_height` instead of estimating.
One object split into several boxes	Raise `eps`; lower `voxel` or `min_size`.
Separate objects merged into one box	Lower `eps`; tighten `max_box`.
Too many tiny noise boxes	Raise `min_size`; raise `min_range`.
No boxes at all	Wrong fixed frame or topic; ROI too small or `roi_z` cuts the ground; `min_size` too high.
Too slow / frames dropped	Raise `voxel`; shrink `roi_x`, `roi_y` or `max_range`; on a Jetson enable max clocks.

5.2 Suggested tuning order

1. Get the geometry right first: topic, frame, `roi_z`, `sensor_height`.
2. Tune the ground filter (`slope_deg`, `min_h`) until ground and obstacles separate cleanly in a view of the obstacle points.
3. Tune clustering (`eps`, `min_size`) once the obstacle points are clean.
4. Finally trade speed for resolution with `voxel` and the ROI.

Offline, use the ground F1 and box-found percentage to compare settings objectively; live, use a looped bag and RViz.

6 Limitations

- **Single frame, no tracking.** Each frame is clustered independently, so box IDs are not stable over time and boxes can flicker.
- **Boxes are partial.** A box is fitted to the visible points only, so it covers the visible side of an object; heading is ambiguous for short, square-ish clusters.
- **Level-sensor assumption.** The ground filter assumes the sensor's x–y plane is roughly parallel to the ground. A tilted mount needs a transform (for example with tf2) into a level frame before `segment()`. This is not implemented.
- **No motion compensation live.** CODa's "comp" clouds are ego-motion compensated; a live cloud usually is not, so fast motion can smear objects slightly.
- **Class-agnostic.** Clusters are not classified; every box is just an obstacle.
- **Density effects.** Far objects have few points and fragment, and `min_size` is counted after downsampling.